Secure File Sharing with S3 and CloudFront

Services: S3, CloudFront, IAM, Lambda@Edge

Description: Implement a secure file-sharing service where users can upload files to S3, and CloudFront serves them with signed URLs.

Skills: CDN setup, secure file access, IAM policies.

Definition: This project implements a secure file sharing system using Amazon S3, CloudFront, and AWS Lambda, enabling controlled access to files stored in the cloud. Users can upload files to a private S3 bucket, and access is granted through signed URLs generated dynamically by a Lambda function. The system ensures that files are served securely via CloudFront, with URLs that are time-limited and cryptographically signed using RSA key pairs.

Scope of the project:

- Enables secure storage of files in Amazon S3 with no public access.
- Provides temporary access to files using CloudFront signed URLs.
- Supports serverless architecture using AWS Lambda to generate signed URLs.
- Ensures fast content delivery worldwide via CloudFront CDN.
- Allows file upload and download in a controlled and auditable manner.
- Implements role-based access control using IAM policies and permissions.
- Provides scalability and reliability without managing servers.
- Can be extended to different file types, multiple users, and automated workflows.

Methodology:

1.Lambda + CloudFront Signed URL (Implemented Method)

- Uses Lambda to generate signed URLs and CloudFront for secure content delivery.
- Ensures files in S3 remain private and cannot be accessed directly.
- Temporary signed URLs allow controlled access for a limited duration.
- Provides high security, suitable for sensitive or restricted files.
- Slightly complex setup requiring RSA key pair and Lambda configuration.

2.S3 Pre-Signed URL (Direct Method)

- Uses S3 pre-signed URLs to allow temporary access to private files.
- Quick and simple to implement using Boto3 or AWS SDK.
- Users can download or upload files for a limited time.
- Ideal for small-scale, internal, or temporary projects.
- Less secure for large-scale or public-facing applications compared to CloudFront signed URLs.

AWS Services Used in the Project:

1,Amazon S3 (Simple Storage Service)

- **Definition:** A scalable object storage service for storing and retrieving any amount of data.
- **Role:** Stores all files securely in private buckets.
- **Purpose:** Provides a durable and secure location for files that need controlled access.

2.AWS Lambda

- **Definition:** A serverless compute service that runs code in response to events without provisioning servers.
- **Role:** Generates signed URLs for secure access to files stored in S3.
- **Purpose:** Enables dynamic, programmatic access control and URL expiration without managing servers.

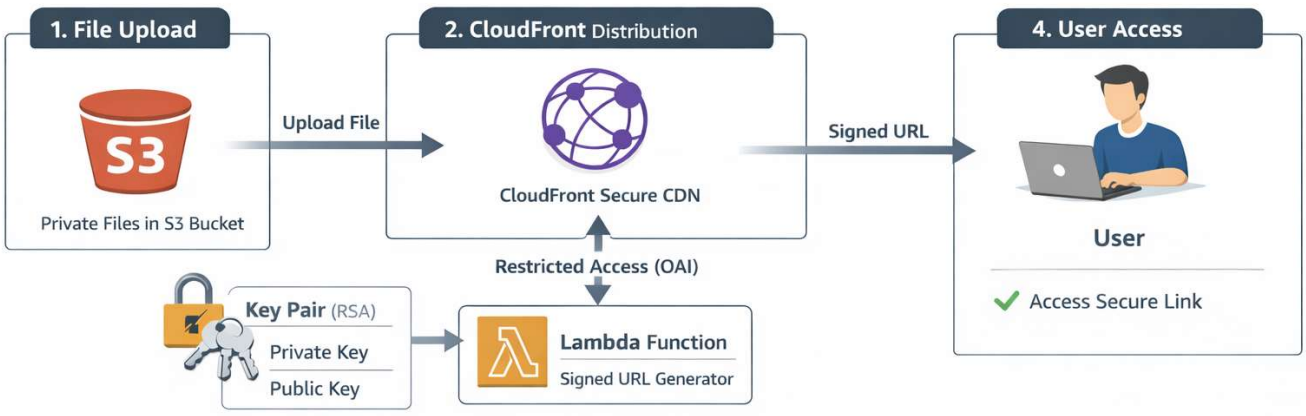
3.Amazon CloudFront

- **Definition:** A content delivery network (CDN) that delivers data, videos, and applications globally with low latency.
- **Role:** Distributes files securely to users using signed URLs.
- **Purpose:** Ensures fast and secure delivery of private content over the internet.4.

4.IAM (Identity and Access Management)

- **Definition:** A service for managing access to AWS resources securely.
- **Role:** Manages permissions for Lambda, S3, and CloudFront.
- **Purpose:** Controls which users and services can access files, ensuring security and least privilege.

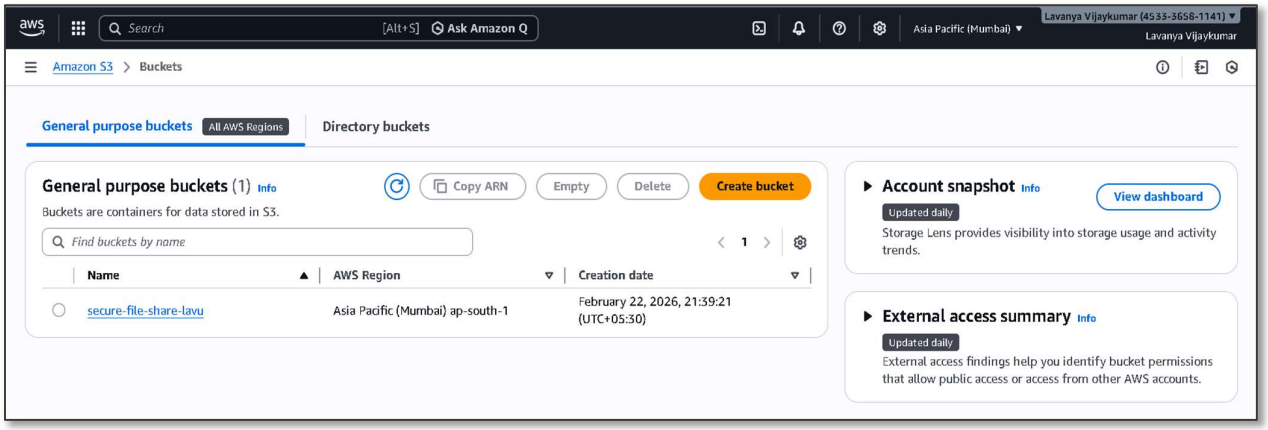
Architecture Diagram:



Project Implementation Steps:

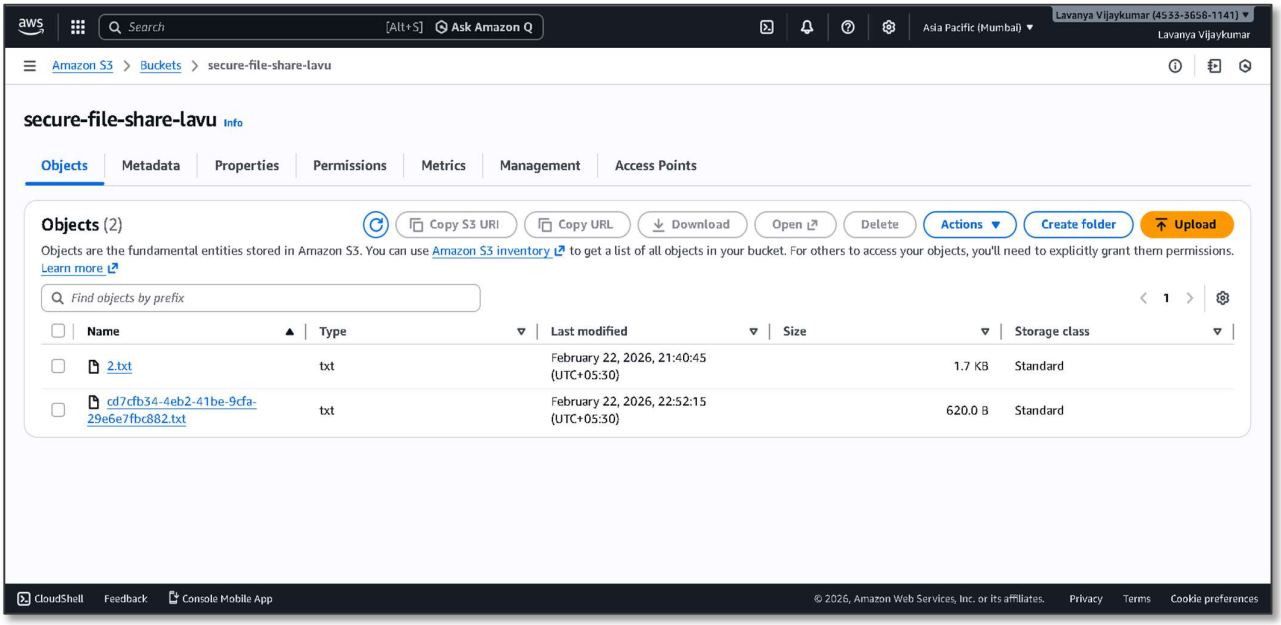
STEP 1:Create S3 Bucket

- Go to AWS S3 → Create bucket.
- Name it secure-file-share-lavu.
- Region: ap-south-1.
- Block all public access.-enable
- Create the bucket.



STEP 2:Upload Sample File

- Upload a test file, e.g., 2.txt.
- Ensure Block Public Access is ON.
- This file will later be accessed via CloudFront signed URL.



STEP 3:Manual RSA key generation:

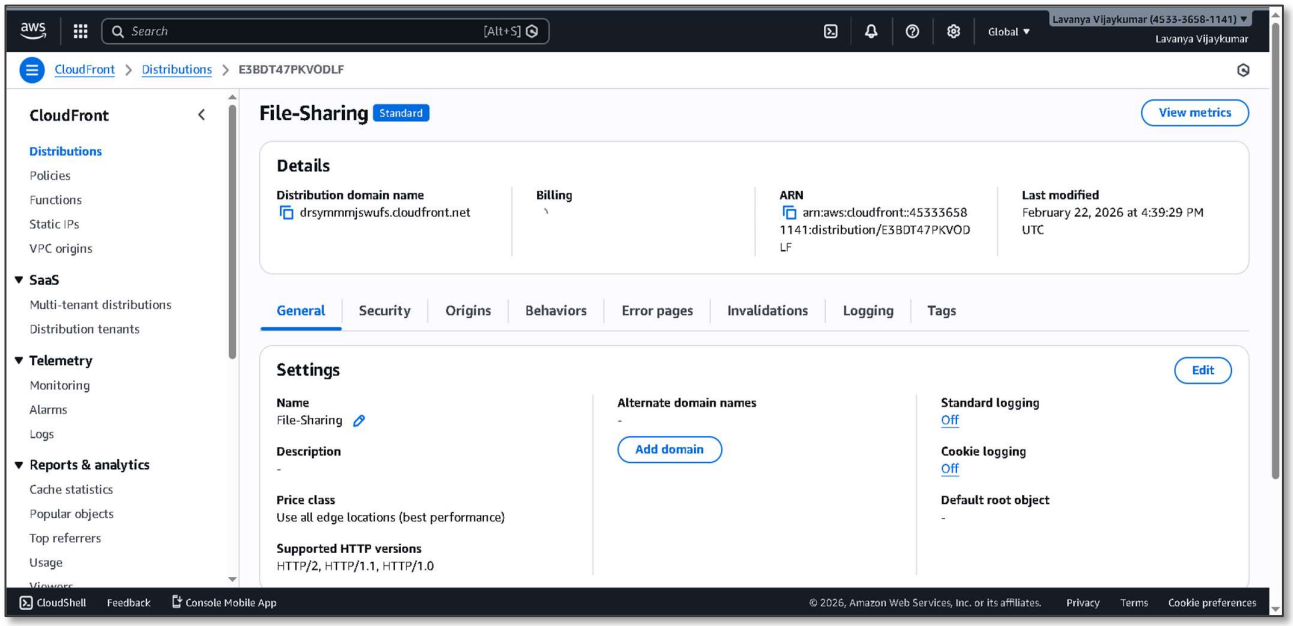
```
# Generate a 2048-bit RSA private key
openssl genrsa -out private_key.pem 2048

# Extract public key from private key
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

- Keep private_key.pem safe; it is never uploaded publicly.
- The public key must be uploaded to CloudFront to register it.

STEP 4 Set Up CloudFront Distribution

- Go to **AWS CloudFront → Create Distribution → Web**.
- Set **Origin Domain** as your S3 bucket (secure-file-share-lavu.s3.ap-south-1.amazonaws.com).
- Set **Restrict Bucket Access → Yes**.
- Create a new **Origin Access Identity (OAI)**.
- Update the bucket policy automatically via CloudFront (allows OAI to read objects).
- Set default cache behavior → **Viewer Protocol Policy → Redirect HTTP to HTTPS**.
- Create the distribution.



STEP 5:Prepare Lambda Function for Signed URL

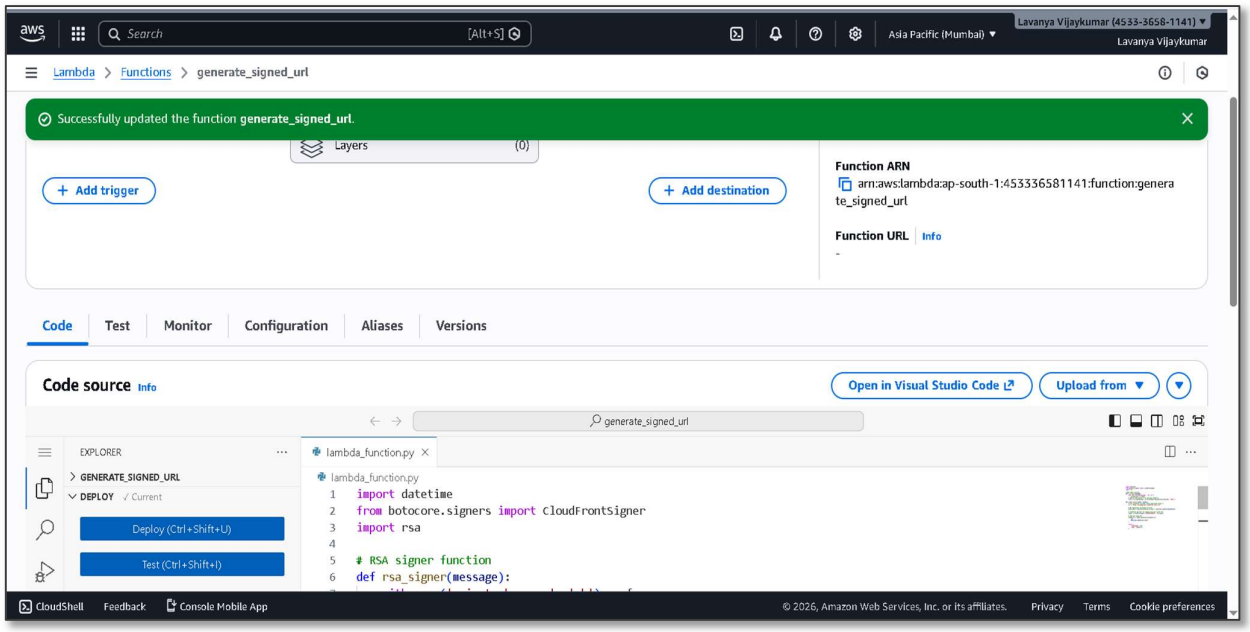
- Create a folder locally: lambda_signed_url.
- Place the following inside:
- lambda_function.py → Python code to generate signed URLs.
- private_key.pem → Private key from CloudFront key pair.

lambda_signed_url/

├── lambda_function.py

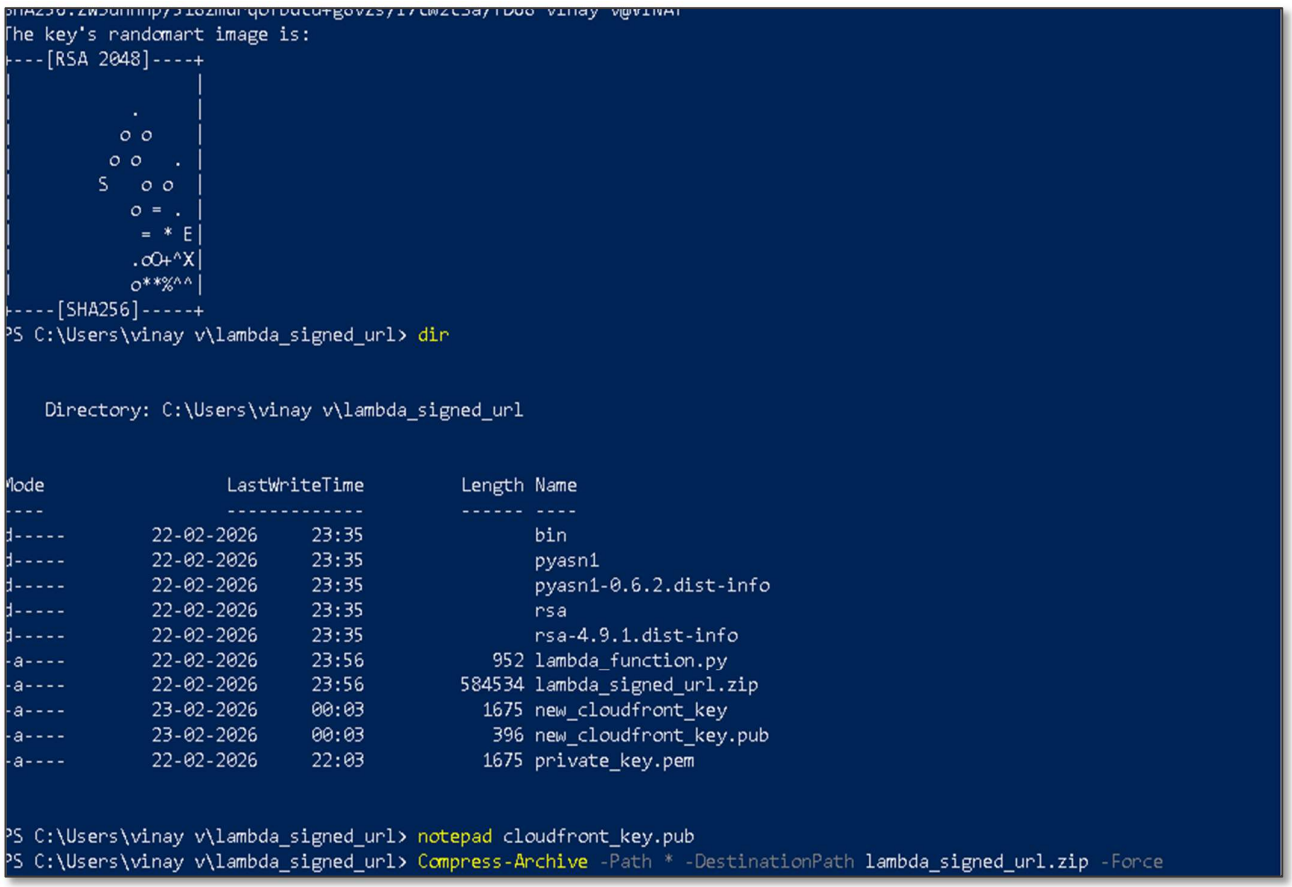
└── private_key.pem

- Update lambda code in lambda_fucntion.py



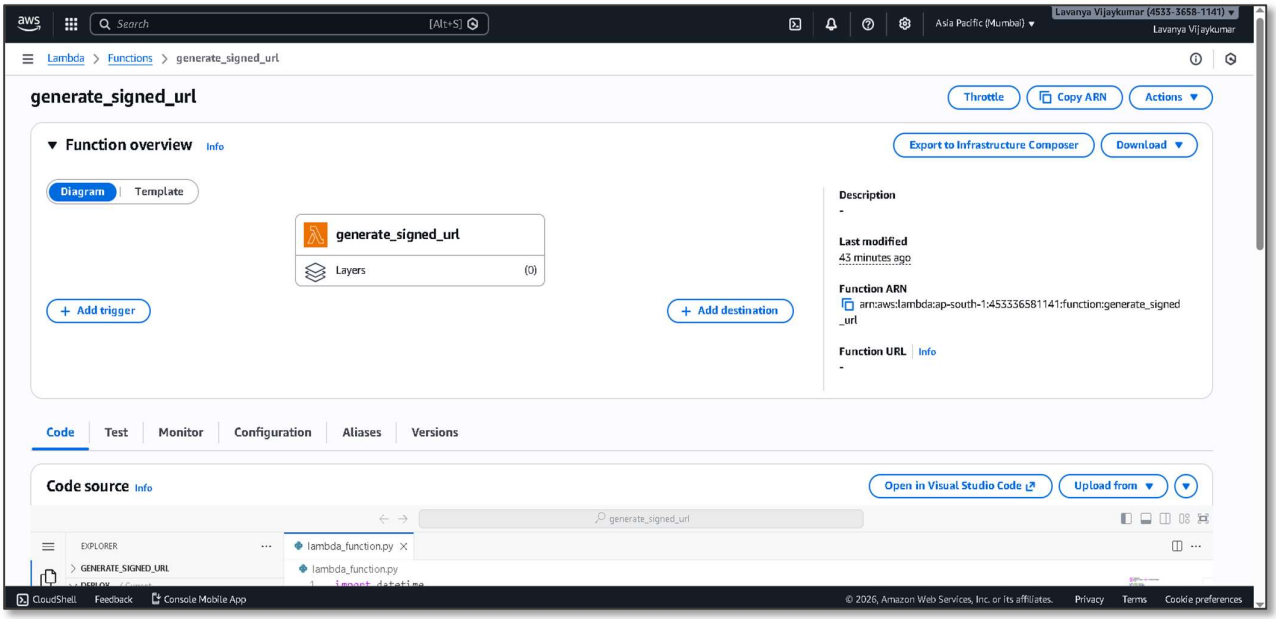
STEP 6:Create ZIP for Lambda

- Open PowerShell in the folder: lambda_signed_url.
- Run:
- Compress-Archive -Path * -DestinationPath lambda_signed_url.zip -Force
- Force overwrites any existing ZIP.



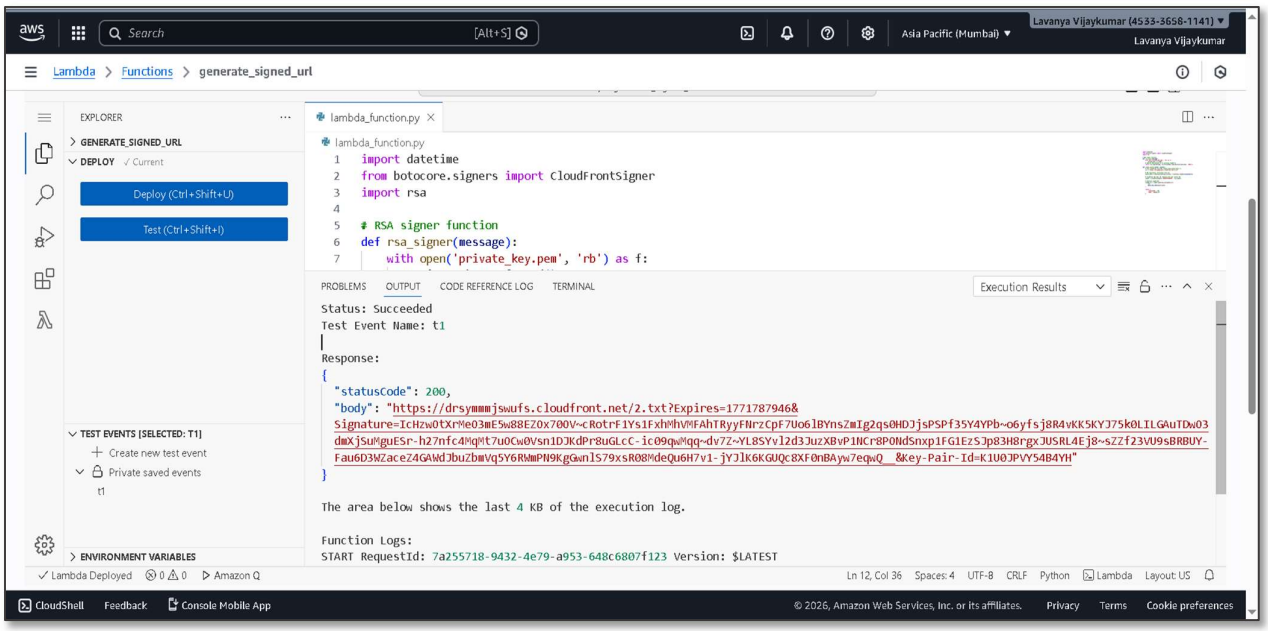
Step 7 Deploy Lambda Function

- Go to AWS Lambda → Create Function → Author from scratch.
- Name: generate_signed_url.
- Runtime: Python 3.9 (or latest supported).
- Upload ZIP: lambda_signed_url.zip.
- Set Handler: lambda_function.lambda_handler.
- Increase timeout to 10 seconds (optional).



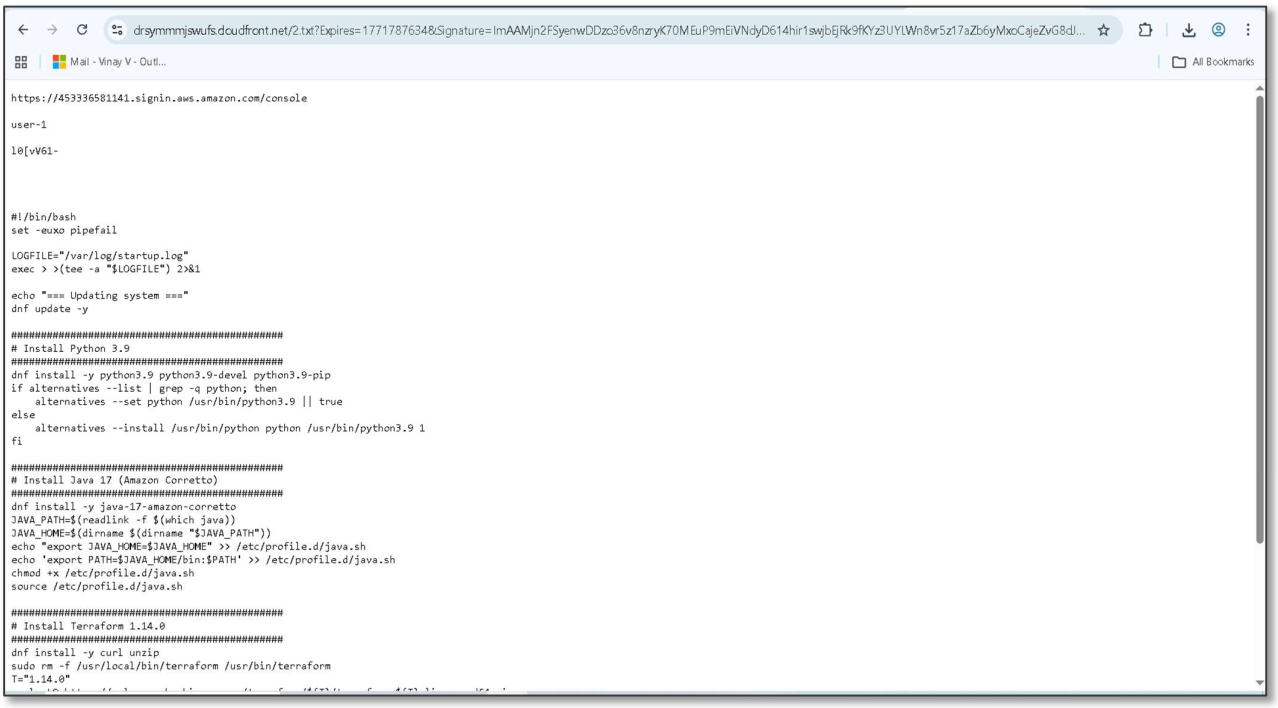
STEP 8:Lambda Function

- Create a test event:
- Run test → Check the body in the response.
- It will return a **signed URL** like:



STEP 8: Access File via Signed URL

- Copy the signed URL from Lambda response.
- Open in browser



Troubleshooting:

Issue 1: Key format errors when uploading public key (Unknown/Invalid Key)

Cause: CloudFront requires the PEM format, not the SSH format.

Solution: Convert your public key to PEM format with -----BEGIN PUBLIC KEY----- and -----END PUBLIC KEY----- headers**.**

Issue 2: MissingKey or Missing Key-Pair-Id query parameter or cookie value

Cause: CloudFront requires a valid Key Pair ID and signed URL for private content. The URL was either unsigned or the Key Pair ID was missing.

Solution: Generate signed URLs using AWS Lambda with the correct CloudFront Key Pair. Ensure the URL includes both the signature and the Key-Pair-Id query parameters.

Issue 3: Unable to import module 'lambda_function': No module named 'rsa'

Cause: The Lambda runtime environment did not have the rsa Python package installed.

Solution: Package your Lambda code along with the rsa library in a deployment ZIP. Use pip install rsa -t . in your project folder before zipping and uploading to Lambda.

Conclusion:

The Secure File Sharing with AWS S3 and CloudFront project successfully demonstrates a secure method to share files over the internet while controlling access. By using pre-signed URLs, only authorized users can access private files, ensuring the confidentiality and integrity of sensitive data. This approach prevents unauthorized access and protects files from public exposure.

Integration of AWS CloudFront enhances content delivery by providing a fast, low-latency experience for users globally. Coupled with AWS Lambda, the generation of signed URLs is automated, reducing manual intervention and minimizing human errors. This serverless approach also ensures scalability, as the system can handle multiple requests without provisioning additional infrastructure.

The project provided hands-on experience with AWS services such as S3, CloudFront, Lambda, and Key Pair Management, helping understand practical cloud security mechanisms. By implementing origin access identities, bucket policies, and RSA key signing, the project emphasizes real-world best practices for secure cloud architectures.

Finally, the project demonstrates a scalable, secure, and flexible architecture suitable for modern applications that require controlled file sharing. It combines cloud security, serverless computing, and content delivery networks to create an efficient and robust solution for file distribution in professional and academic scenarios